T. Kodanda Ram
192524162
CSA0619 - DAA

CO2 - AT1
Analytical Problem Solving.

(39) **Linear Search - Single Match in Middle position**

88%

1) problem understanding

The given dataset is :

A = [14, 28, 35, 47, 59, 63, 71]

The dataset is unsorted, & the task is to search for the target element 47.

Since the data is not arranged in ascending order, or descending order, Binary search cannot be applied. Therefore each element must be checked sequentially from the beginning until the target element is found.

2) Given

Dataset : A = [14, 28, 35, 47, 59, 63, 71]

Target element : key = 47

no. of elements : n = 7

3) Input & output

Input : Dataset = [14, 28, 35, 47, 59, 63, 71]

key = 47

output : Element 47 is found at position 4 (index = 3).

4) Method Selection

Algorithm used : Linear Search (sequential search)

Justification

Linear search is the most suitable algorithm because:

- The dataset is unsorted.
- It searches elements one by one from the beginning.
- No preprocessing or sorting is required.
- It is simple & efficient for small unsorted datasets.

Hence, Linear search is the correct method.

5) Algorithm (pseudo code)

```
LinearSearch (A, n, key)
    for i = 0 to n-1
        if A[i] == key
            return i
    return -1
```

6) step by step solution

Initial dataset

position :	1	2	3	4	5	6	7
Element :	14	28	35	47	59	63	71

step 1: compare 14 ≠ 47 no
move to the next element.

step 2: compare 28 ≠ 47 no
move to the next element

step 3: compare 35 ≠ 47 no
move to the next element

step 4: compare 47 = 47 yes
The required element is found.
The search stops immediately.

7) Comparision Table

comparision	Element checked	Result.
1	14	not found
2	28	not found
3	35	not found
4	47	found.

8) Calculations

Total comparisions

comparision 1 : 14 ≠ 47

comparision 2 : 28 ≠ 47

comparision 3 : 35 ≠ 47

comparision 4 : 47 = 47

∴ Total comparisions = 4

Position of the Target

0-based index = 3

1-based position = 4

9) Time Complexity Analysis

Best Case

The target is the first element

only one comparision is required

Time complexity = O(1)

Average case

The target may appear anywhere in the list.

Approximately n/2 comparisions are required.

Time complexity = O(n)

worst case

The target is the last element or not present.

All elements must be checked

Time complexity = O(n)

10) Space Complexity

Linear search uses only one extra variable to store the index during searching.

space complexity = O(1)

11) Verification of the Result:

The element 47 is present in the dataset.

It is located after checking 4 elements.

The algorithm correctly returns:

- Element found
- position = 4
- Index = 3

Hence, the obtained result is verified to be correct.

13) Interpretation of the result:

Linear search found the element 47 after 4 comparisions. since the target is near the middle of the list, it checked only part of the dataset before finding it Linear search is suitable for small or unsorted datasets but it becomes slower for large datasets because it may need to check every element. The efficiency of Linear search is moderate when the target lies near the middle of the list, as it requires approximately half of the elements to be checked before finding the target.

Result:

The element 47 is successfully found at the 4th position (Index 3) after 4 comparisions using the Linear search algorithm. The algorithm is correct, requires O(1) space, and has O(1), O(n) & O(n) time complexity in the best, average, and worst cases respectively.